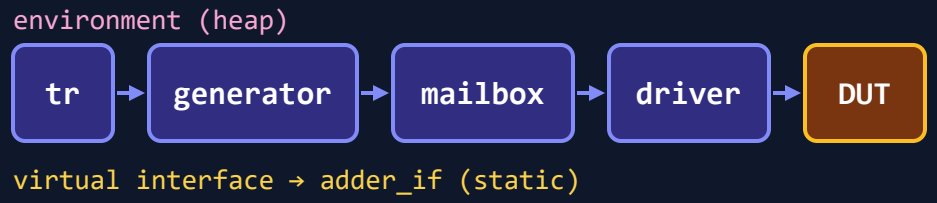
SYSTEMVERILOG · HOMEWORK

SV Adder TB

코드 분석 · 시뮬레이션

메모리 구조 관점으로 본 SystemVerilog 검증 환경



Stack · Heap · bss · Text · 코드 한 줄의 메모리 위치 추적

정안성

2026. 11

구조

generator → mailbox → driver

environment가 new · 연결, virtual interface로 DUT 구동

메모리

Stack · Heap · bss · Text

class 객체 = Heap, adder_if · DUT = 정적

자극

repeat · randomize · put

put 시간 소모 0 → GEN FINISH 0ns

결과

drv.run() forever 대기

get() 대기 → env finish 미출력

Contents

tb_adder
메모리 구조 · 코드 해석 · 시뮬레이션

01 **메모리 구조** p. 3 – 4
프로세스 메모리 · OOP 동적 할당

02 **tb_adder 구조** p. 5 – 7
계층 · 실행 순서 · 구역별 수명 · 확장 구조

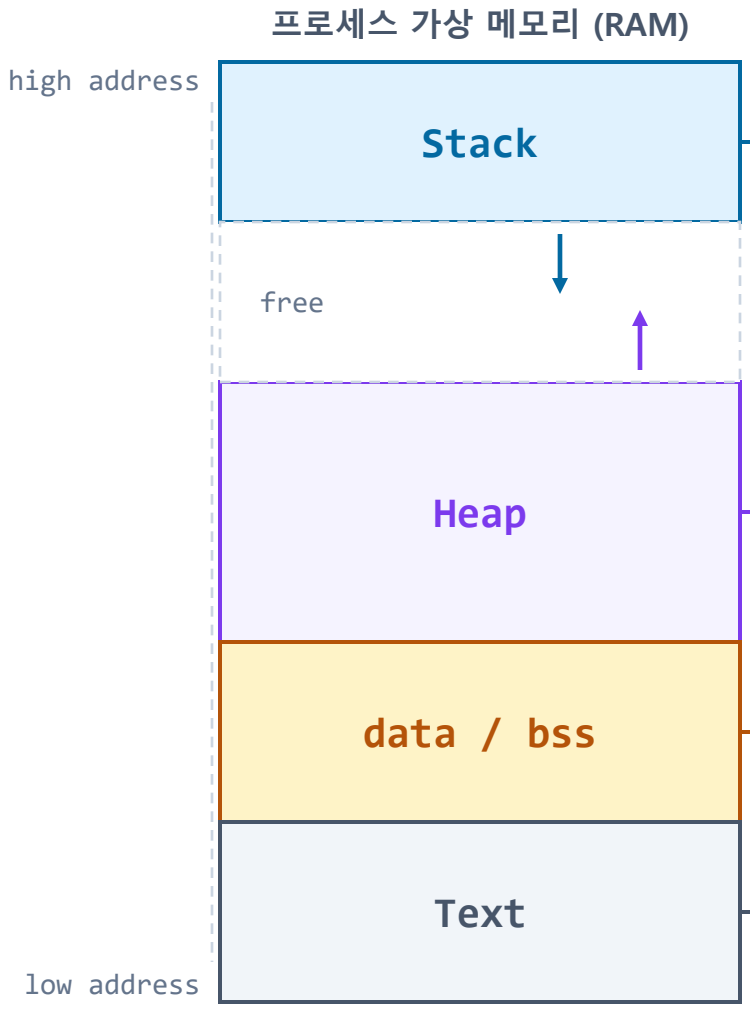
03 **코드 해석** p. 8 – 13
transaction · generator · driver

04 **시뮬레이션** p. 14 – 15
gen.run() 종료 · driver forever 대기

프로세스 메모리: Stack · Heap · bss · Text 네 구역

01

결론 · 지역변수 → Stack · new 객체 → Heap · 전역 · static 변수 → bss (data) · 실행 코드 → Text



1 지역변수
function · task 호출 시 생성 → 반환 시 해제

2 동적 할당 (new)
객체 생성 시 할당 → 참조 0개일 때 GC 회수

3 전역 · static 변수
프로그램 시작 시 할당 → 종료까지 유지

4 실행 코드 (read-only)
class의 function · task 코드 · class별 1벌

OOP 검증: new 시점에 Heap 할당, 참조 0개일 때 GC 회수

01

결론 · Verilog 정적 할당 → 시뮬레이션 끝까지 메모리 점유 · SV class → new 시에만 Heap 할당
· handle 선언만으로는 null → new가 객체의 가상 주소를 handle에 저장

Verilog 검증 — 정적 할당
실행 순간 변수가 고정 주소에 할당
→ 시뮬레이션 종료까지 리소스 점유

SystemVerilog OOP — 동적 할당
생성 전에는 메모리 할당 없음
→ new 시 heap에 할당, 참조가 사라질 경우 GC가 회수

handle → 객체 생성 흐름

handle 선언
transaction tr; → null

→ **new**
객체 생성

→ **가상 주소 할당**
OS가 시뮬레이터에 할당

→ **page table**
OS가 생성 · 관리

→ **MMU**
r/w 시 page table 참조
→ 물리 주소 변환

시뮬레이터 가상 공간 중 heap (예시)

가상 주소	값
0x001	**0x5000**

tr handle

가상 주소	값
0x5000	헤더 : 타입 정보
0x5008	tr.a 값
0x5009	tr.b 값

tr 객체

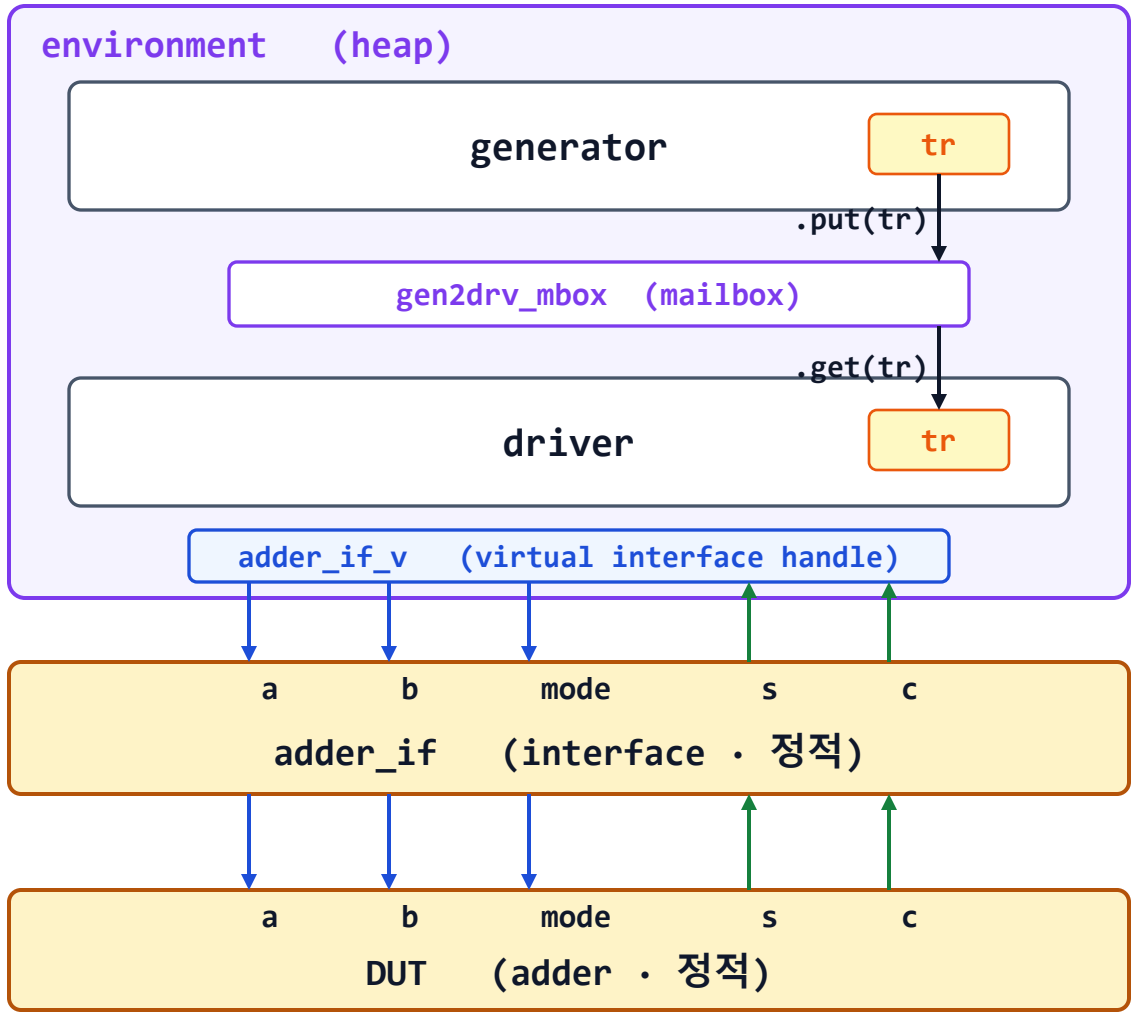
new
1. 객체 생성 시점에 객체의 가상 주소 할당
→ 그 주소 아래로 멤버 변수 공간 할당

MMU
2. 시뮬레이터가 보는 주소 = 가상 주소
→ 실제 위치는 OS의 page table + MMU가 결정

tb_adder: Heap의 class 객체가 virtual interface로 정적 DUT 구동

02

결론 · environment · generator · driver · mailbox → **Heap** (new 시 생성)
· adder_if · DUT → **정적** (elaboration 시 고정) → class는 adder_if_v (virtual) 참조로만 접근



tb_adder 실행 순서

1 elaboration
adder_interface adder_if(); adder dut(...);
→ 정적 영역에 고정, env handle = null

2 0ns initial
env = new(adder_if);
→ heap에 env 생성, 내부에서 mbox → gen → drv 순서로 new

3 env.run()
gen.run() : tr 100개 생성 → mailbox에 put
drv.run() : get → virtual interface → DUT

new() 실행 시

1. 시뮬레이터 프로세스의 heap에 객체 공간 할당
2. 멤버 기본값 초기화 (bit = 0, logic = x) → constructor 실행
3. 객체 참조값 (주소)을 handle 변수에 저장
(주소는 실행마다 다를 수 있음)

메모리 구역별 생성 · 소멸 시점과 tb_adder 대응

02

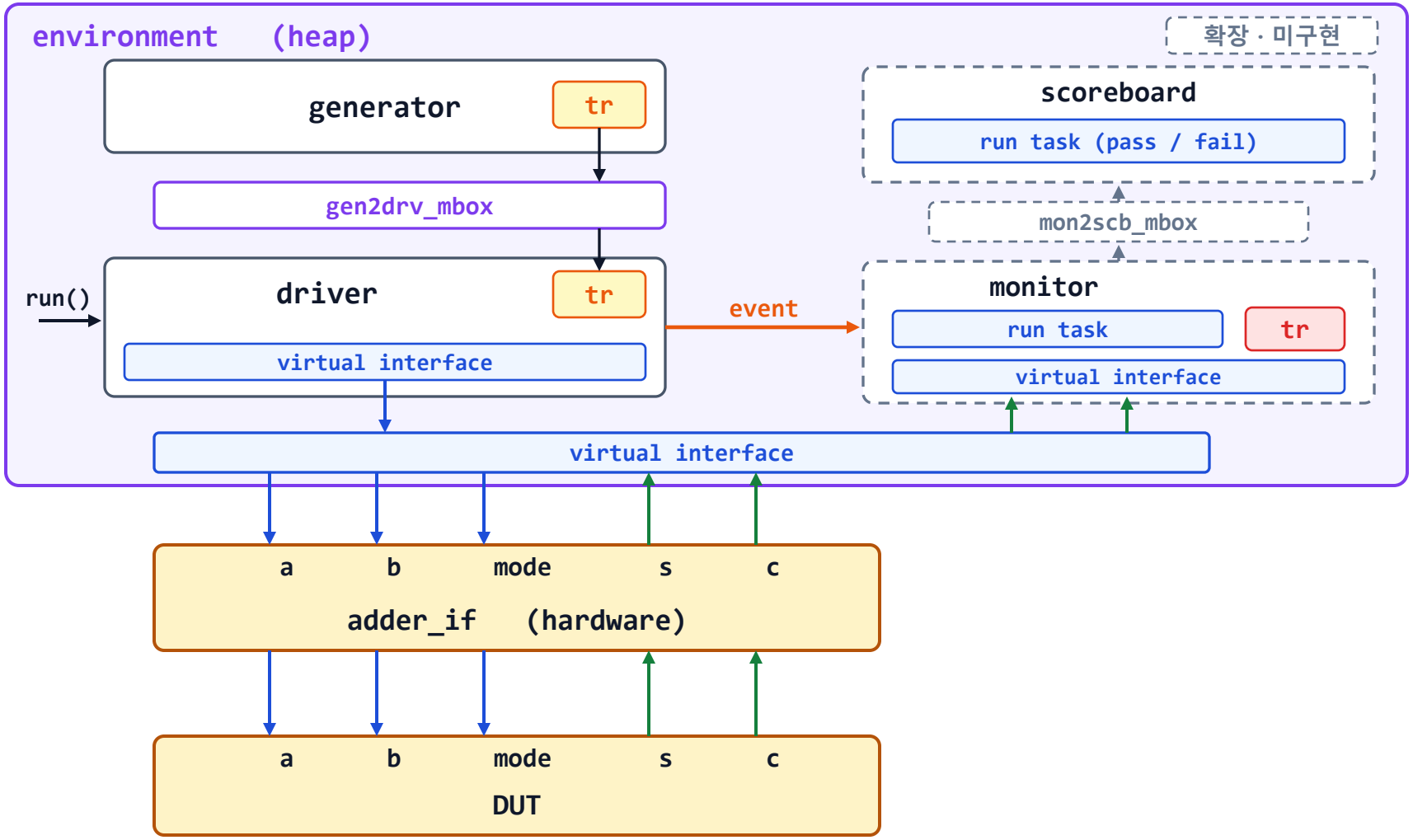
결론 · class 코드 → Text · 인스턴스 · module 변수 → bss · 지역변수 → Stack · 객체 → Heap (참조 0개일 때 GC)

구역	생성 시점	소멸 시점	tb_adder 대응
Text	프로그램 로드 시	프로그램 종료 시	class의 function · task 코드 (class별 1벌)
Data / bss (static)	시뮬 시작 (elaboration)	시뮬 종료	adder_if · dut 인스턴스 module 변수 env handle
Stack	function · task 호출 시	반환될 때	new() 인자 · run() 지역변수
Heap	new() 실행 시	참조 handle 0개일 때 (GC)	env · gen · drv · mailbox · tr

02

확장 구조: monitor · scoreboard도 Heap, adder_if · DUT는 정적

결론 · 현재 tb_adder = generator → driver까지 · monitor · scoreboard · mon2scb_mbox는 확장 구조
· Stack = 인자 · 실행 프레임 · Heap = 객체 · 큐 · bss = 인스턴스 · module 변수 · Text = class 함수



PC MEMORY

STACK
지역변수 할당
· new() 인자
(gen2drv_mbox_a, adder_if_v_a)
· run() 실행 프레임

Heap
동적 할당 / 배열
· env · gen · drv
· mailbox (주소 큐)
· tr × 100 (new마다 1개)

bss
정적 할당 (data)
· adder_if · dut 인스턴스
· module 변수 env handle
(null → env 객체 주소)

Text
코드 영역
· class별 함수 1벌
run · new · debug_print

transaction: rand bit 입력 · logic 출력 묶음, new 시 Heap에 생성

03

결론 · class = 설계도 → 컴파일 후 Text 영역에 로드, 실행 시점에는 **handle = null (객체 없음)**
· 멤버 변수 → new 시 heap의 tr 객체 · debug_print → Text 영역에 class별 1벌

```
class transaction; // -> 벡터, 스티뮬러스를 생성하기위한 사>
용자 객체  //동적할당이 가능하다

    //drive
    // rand : randomize() 기능 - 'rand' KeyWord 있는 변수만
random값을 생성한다.
1
    rand bit [7:0] a;
    rand bit [7:0] b;
    rand bit       mode;


    //monitor - 출력 값은 4상태가 나올 수 있으므로, logic 선
언
    logic [7:0]  s;
    logic        c;


2
    function debug_print(string name);

    $display("%t :[%s] a =%d, b = %d, mode = %d, s = %d, c =
%d",$time,name,a,b,mode,s,c);


    endfunction


endclass
```

1 `rand bit a · b · mode` / `logic s · c`

new 시 heap의 tr 객체 멤버로 저장 · new 직후 heap의 tr 객체 (예시 주소)

0x5000	0x5008	0x5009	0x500A	0x500B-C	0x500D
헤더 : 타입 정보	a = 0	b = 0	mode = 0	s = x	c = x

- **bit (2상태)** → 0 초기화 · drive용 rand 변수
- **logic (4상태)** → x 초기화 · monitor용, bit당 2 bit 사용 (s 8 bit → 2 byte)

2 `debug_print()`

function 코드 → Text 영역
class별 1벌 · 객체마다 복사 없음

`s · c` (logic)

tr.s · tr.c 값 대입 경로 없음 (monitor 미구현)
→ debug_print 출력 **s = x, c = x**

generator: mailbox 주소를 받아 저장, repeat마다 tr 새로 생성

03

결론 · transaction · mailbox handle 선언 → new(mailbox) 시 **외부 mailbox 주소를 내부 handle에 복사**
· **repeat마다 tr = new → randomize → debug_print → put**

```
1  class generator;

       transaction tr;
       mailbox #(transaction) gen2drv_mbox;


2      function new(mailbox #(transaction) gen2drv_mbox_a);// argument
           gen2drv_mbox = gen2drv_mbox_a;
       endfunction


       task run();
           // tr의 랜덤 스티뮬러스 - class 기본제공 function
           //dr -> a,b를 어떻게 드라이브 할 것인가. -> 가상 인터페이스로 >
연결해야한다. 이건 SW 따라서 Virtual 키워드를 써야한다.

           repeat(100) begin
3              tr = new;
               tr.randomize();
               tr.debug_print("GEN");
               gen2drv_mbox.put(tr);
               end

       endtask


endclass
```

1 `tr · gen2drv_mbox`
transaction · mailbox 객체 handle
new 시 handle에 참조 주소 저장 (시뮬레이터가 heap에 생성)

2 `new(gen2drv_mbox_a)`
생성 시 외부 mailbox 객체의 가상 주소를 내부 handle에 복사

외부 객체 = environment가 만든 mailbox
`env.new : gen2drv_mbox = new;  →  gen = new(gen2drv_mbox);`
순서 반대일 경우 null 복사 → put 시 오류

3 `tr = new`
repeat마다 새 객체 생성 → handle tr에 주소 저장

new
tr →

가상 주소	값
0x001	**0x5000**
.	.
0x5000	헤더 : 타입 정보
0x5008	tr.a 값
0x5009	tr.b 값

→ 생성 시 **bit = 0, logic = x**

generator run: randomize로 값 갱신, put은 tr 주소만 mailbox에 저장

03

결론 · randomize → heap의 tr.a · tr.b 값 갱신 · put → tr 주소 (0x5000)를 mailbox 큐에 저장
· put 시간 소모 0 → 0ns에 100개 put, tr 100개가 heap에 동시 존재

```
repeat(100) begin
tr = new;
1 tr.randomize();
tr.debug_print("GEN");
2 gen2drv_mbox.put(tr);
end

endtask
```

객체 내부 함수 code (Text) 영역에 저장 · class별 별도 코드 공간

tr = new를 repeat 밖에 둘 경우
객체 1개 (0x5000)에 randomize만 덮어씀 → mailbox에 0x5000 × 100
→ driver는 마지막 값만 100번 수신

1 randomize — heap의 tr 객체 값 갱신

new
tr →

가상 주소	값
0x001	**0x5000**
.	.
.	.
0x5000	헤더 : 타입 정보
0x5008	tr.a 값 = 0
0x5009	tr.b 값 = 0
.	.

randomize()
→ tr.a 값 = 5
→ tr.b 값 = 90

시뮬레이터 내 가상 공간 중 heap 공간 (예시 주소)

2 put — mailbox 객체 (0x9000) 내부 큐

gen2drv_mbox.put(tr) →

가상 주소	값
0x9000	**0x5000**
0x9008	0x6000
.	.
.	.

repeat 1 : tr = new → 0x5000
repeat 2 : tr = new → 0x6000 (새 객체)

큐 저장 값 = tr의 값 = **transaction 객체 주소**

put 시간 소모 0 → 0ns에 100개 적재 (tr 100개 heap에 동시 존재)

driver: virtual interface handle로 정적 adder_if 참조

03

결론 · class 객체 = new · GC로 생성 · 소멸, interface = 시작 전 고정 → **class는 instance 참조만 가능 (virtual)**
· adder_if = 정적 영역의 HW 1개 → 모든 vif handle이 같은 실체를 가리킴

```
class driver;
    transaction tr;    // generator가 저장한 메모리 주소를 가져와야 하기 때문에 핸들러가 있어야 한

    mailbox #(transaction) gen2drv_mbox;

    virtual adder_interface adder_if_v; // interface는 하드웨어이기 때문에 virtual 핸들링
    function new(mailbox #(transaction) gen2drv_mbox_a,  virtual adder_interface adder_if_v_a);//
) 안에는 실제 인터페이스가 들어와야함

        adder_if_v = adder_if_v_a; // 실제 하드웨어로 연결 ( connection)
        gen2drv_mbox = gen2drv_mbox_a;

    endfunction

    task run();
        // tr의 랜덤 스티뮬러스 - class 기본제공 function
        //dr -> a,b를 어떻게 드라이브 할 것인가. -> 가상 인터페이스로 연결해야한다. 이건 SW 따라서
Virtual 키워드를 써야한다.

        forever begin
            gen2drv_mbox.get(tr);
            tr.debug_print("DRV");

            adder_if_v.a = tr.a;
            adder_if_v.b = tr.b;
            adder_if_v.mode = tr.mode;


            #10;

        end

    endtask


endclass
```

1 **tr · gen2drv_mbox** transaction handle · mailbox handle

2 **virtual adder_interface adder_if_v**
class 객체 = 동적 할당 → new로 생성, GC로 소멸
interface = 정적 → 시뮬레이션 시작 전 구조 고정
→ class는 이미 존재하는 instance를 가리키는 참조만 가능

시점	vif 상태
`virtual adder_interface adder_if_v;`	null
`adder_if_v = adder_if_v_a;` (constructor 인자)	실제 instance 참조
연결 없이 `adder_if_v.a = 1;`	**실행 중 null 접근 오류**

실체 interface 경로 — tb_adder (top)

tb_adder
adder_interface adder_if();
→ **initial**
env = new(adder_if);
→ **env.new**
drv = new(mbox, adder_if_v);
→ **drv 내부**
adder_if_v = adder_if 참조

driver new: 외부 mailbox · interface 주소를 내부 handle에 복사

03

결론 · driver 생성 시 참조 객체 = mailbox · virtual interface (environment가 생성한 외부 객체)
· 참조 = 객체의 가상 주소 복사 → gen · drv 모두 같은 mailbox (0x9000)

```
class driver;

    transaction tr;    // generator가 저장한 메모리 주소를 가져와야 하기 때문에 핸들러가 있어야 한다

    mailbox #(transaction) gen2drv_mbox;


    virtual adder_interface adder_if_v; // interface는 하드웨어이기 때문에 virtual 핸들링
    function new(mailbox #(transaction) gen2drv_mbox_a,  virtual adder_interface adder_if_v_a);//
) 안에는 실제 인터페이스가 들어와야함

1       adder_if_v = adder_if_v_a; // 실제 하드웨어로 연결 ( connection)
2       gen2drv_mbox = gen2drv_mbox_a;

    endfunction


    task run();
        // tr의 랜덤 스티뮬러스 - class 기본제공 function
        //dr -> a,b를 어떻게 드라이브 할 것인가. -> 가상 인터페이스로 연결해야한다. 이건 SW 따라서
Virtual 키워드를 써야한다.

        forever begin
            gen2drv_mbox.get(tr);
            tr.debug_print("DRV");

            adder_if_v.a = tr.a;
            adder_if_v.b = tr.b;
            adder_if_v.mode = tr.mode;


            #10;

        end

    endtask


endclass
```

1 `adder_if_v = adder_if_v_a` 외부 interface → 내부 vif handle이 참조

2 `gen2drv_mbox = gen2drv_mbox_a` 외부 mailbox → 내부 handle이 참조

handle 값 복사 (driver 생성 시)

	가상 주소	값
env.gen2drv_mbox	0x2018	0x9000
	.	.
	.	.
drv.gen2drv_mbox	0x10000	0x9000
	.	.

0x9000 복사

내부 handle이
외부 객체 참조 값 복사

```
// environment::new — gen · drv 모두 같은 mailbox (0x9000) 주소 수신
gen2drv_mbox = new;  gen = new(gen2drv_mbox);
drv = new(gen2drv_mbox, adder_if_v);
```

driver run: get으로 tr 주소 수신 → virtual interface로 DUT 입력

03

결론
- get(tr) → 큐 맨 앞 주소 (0x5000)를 drv.tr에 복사, 해당 slot은 큐에서 제거
- adder_if_v.a = tr.a → heap의 값이 실체 adder_if.a로 복사 → DUT 입력

```
class driver;

    transaction tr;    // generator가 저장한 메모리 주소를 가져와야 하기 때문에 핸들러가 있어야 한

    mailbox #(transaction) gen2drv_mbox;


    virtual adder_interface adder_if_v; // interface는 하드웨어이기 때문에 virtual 핸들링
    function new(mailbox #(transaction) gen2drv_mbox_a,  virtual adder_interface adder_if_v_a);//
) 안에는 실제 인터페이스가 들어와야함

        adder_if_v = adder_if_v_a; // 실제 하드웨어로 연결 ( connection)
        gen2drv_mbox = gen2drv_mbox_a;

    endfunction


    task run();
        // tr의 랜덤 스티뮬러스 - class 기본제공 function
        //dr -> a,b를 어떻게 드라이브 할 것인가. -> 가상 인터페이스로 연결해야한다. 이건 SW 따라서
Virtual 키워드를 써야한다.

    forever begin
        gen2drv_mbox.get(tr);
        tr.debug_print("DRV");

        adder_if_v.a = tr.a;
        adder_if_v.b = tr.b;
        adder_if_v.mode = tr.mode;

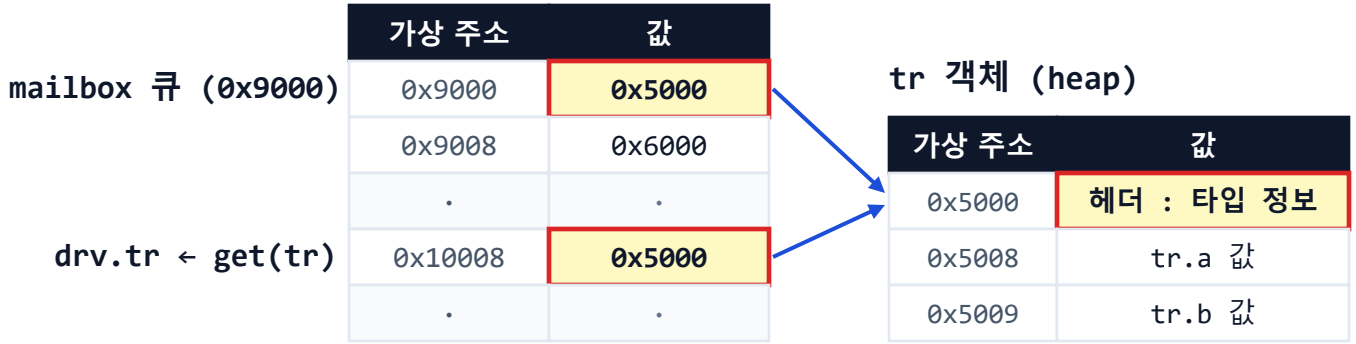

        #10;

    end

    endtask


endclass
```

1
2
3



1. **get(tr)** 큐 맨 앞 주소 → drv.tr 복사, slot 제거
2. **debug_print** 현재 handle 주소에 있는 tr의 함수 실행
3. **adder_if_v.a = tr.a** virtual interface → 실체 adder_if.a → DUT

env.run() = forever { gen.run(); drv.run(); } 시간 흐름

0ns	0 ~ 990ns	1000ns
GEN × 100 heap에 tr 100개	DRV × 100 (#10) 이전 tr 참조 0 → GC	큐 비어 get() 대기 $finish 미도달

04

gen.run(): 256개 put이 0ns에 끝난 뒤 driver 시작

결론 · repeat(100) → repeat(256) + GEN FINISH 출력 추가 → put 시간 소모 0 → GEN FINISH, time 0ns
· GEN 256개 출력 후 DRV가 0ns부터 10ns 간격으로 get

class generator

```
class generator;

    transaction tr;
    mailbox #(transaction) gen2drv_mbox;



    function new(mailbox #(transaction) gen2drv_mbox_a);// argument
        gen2drv_mbox = gen2drv_mbox_a;
    endfunction


    int gen_cnt;
    task run();
        // tr의 랜덤 스티뮬러스 - class 기본제공 function
        //dr -> a,b를 어떻게 드라이브 할 것인가. -> 가상 인터페이스로 연결해야한다. 이건 SW 따라서 Virtual 키워드를 써야한다.

        repeat(256) begin
        tr = new;
        tr.randomize();
        tr.debug_print("GEN");
        gen2drv_mbox.put(tr);
        end
        $display("==========================================");
        $display("GEN FINISH, time %t",$time);
        $display("==========================================");
    endtask


endclass
```

1 **repeat(256)** put × 256 → 0ns에 전부 큐에 적재 → GEN FINISH 출력

2 **GEN FINISH, time 0ns** 다음 줄부터 [DRV] 0ns · 10ns · 20ns …

xsim log

```
0ns :[GEN] a = 81, b = 130, mode = 0, s =   x, c = x
0ns :[GEN] a =142, b = 246, mode = 0, s =   x, c = x
0ns :[GEN] a =148, b =  54, mode = 1, s =   x, c = x
0ns :[GEN] a =165, b = 149, mode = 1, s =   x, c = x
0ns :[GEN] a =192, b = 182, mode = 1, s =   x, c = x
0ns :[GEN] a = 87, b =  92, mode = 0, s =   x, c = x
0ns :[GEN] a =141, b = 219, mode = 0, s =   x, c = x
0ns :[GEN] a =128, b =  56, mode = 1, s =   x, c = x
0ns :[GEN] a =  0, b = 118, mode = 1, s =   x, c = x
0ns :[GEN] a =148, b =  14, mode = 1, s =   x, c = x
0ns :[GEN] a =111, b =  37, mode = 0, s =   x, c = x
0ns :[GEN] a = 75, b =  67, mode = 0, s =   x, c = x
==========================================
GEN FINISH, time 0ns
==========================================
0ns :[DRV] a =189, b = 221, mode = 1, s =   x, c = x
10ns :[DRV] a =173, b = 134, mode = 0, s =   x, c = x
20ns :[DRV] a =170, b =  42, mode = 1, s =   x, c = x
30ns :[DRV] a = 41, b = 248, mode = 1, s =   x, c = x
40ns :[DRV] a =122, b =   5, mode = 0, s =   x, c = x
50ns :[DRV] a =244, b = 236, mode = 0, s =   x, c = x
60ns :[DRV] a =236, b =  88, mode = 1, s =   x, c = x
70ns :[DRV] a = 68, b = 245, mode = 0, s =   x, c = x
80ns :[DRV] a =184, b =  53, mode = 1, s =   x, c = x
90ns :[DRV] a =211, b =  61, mode = 1, s =   x, c = x
100ns :[DRV] a =211, b =  88, mode = 1, s =   x, c = x
110ns :[DRV] a =130, b = 247, mode = 0, s =   x, c = x
```

04

drv.run(): forever 안에서 get() 대기 → env finish 미출력

결론
- gen.run() 0ns 종료 → drv.run() 진입 → forever에 return 없음 → env.run()의 forever 이후 줄 도달 불가
- 마지막 DRV 2550ns → 큐 비어 get() 대기 → 이벤트 소진으로 종료, env finish 미출력 · $finish 미도달

class driver

```
class driver;

    transaction tr;   // generator가 저장한 메모리 주소를 가져와야 하기 때문에 핸들러가 있어야 함
    mailbox #(transaction) gen2drv_mbox;


    virtual adder_interface adder_if_v; // interface는 하드웨어이기 때문에 virtual 핸들링
    function new(mailbox #(transaction) gen2drv_mbox_a,  virtual adder_interface adder_if_v_a);

        adder_if_v = adder_if_v_a; // 실제 하드웨어로 연결 ( connection)
        gen2drv_mbox = gen2drv_mbox_a;

    endfunction



    int drv_cnt;
    task run();
        // tr의 랜덤 스티뮬러스 - class 기본제공 function
        // drv -> a,b들 어떻게 드라이브 할 것인가. -> 가상 인터페이스로 연결해야한다. 이건 SW 따라
        forever begin
            gen2drv_mbox.get(tr);
            tr.debug_print("DRV");

            adder_if_v.a = tr.a;
            adder_if_v.b = tr.b;
            adder_if_v.mode = tr.mode;
            drv_cnt ++ ;

            #10;

        end
    endtask


endclass
```

1

forever
return 없음
get() 대기에서 정지

env finish
forever 다음 줄
구조상 도달 불가

class environment

```
class environment;


    generator gen;
    driver drv;
    mailbox #(transaction) gen2drv_mbox;


    function new(virtual adder_interface adder_if_v);

        gen2drv_mbox = new;
        gen = new(gen2drv_mbox);
        drv = new(gen2drv_mbox, adder_if_v);

    endfunction


    task run();

        forever begin

            gen.run();
            drv.run();

        end
        #10;
        $display("env finish");

    endtask

endclass
```

2

미출력

xsim log

```
0ns :[GEN] a =141, b = 219, mode = 0, s =   x, c = x
0ns :[GEN] a =128, b =  56, mode = 1, s =   x, c = x
0ns :[GEN] a =  0, b = 118, mode = 1, s =   x, c = x
0ns :[GEN] a =148, b =  14, mode = 1, s =   x, c = x
0ns :[GEN] a =111, b =  37, mode = 0, s =   x, c = x
0ns :[GEN] a = 75, b =  67, mode = 0, s =   x, c = x
=========================================
GEN FINISH, time 0ns
=========================================
0ns :[DRV] a =189, b = 221, mode = 1, s =   x, c = x
10ns :[DRV] a =173, b = 134, mode = 0, s =   x, c = x
20ns :[DRV] a =170, b =  42, mode = 1, s =   x, c = x
30ns :[DRV] a = 41, b = 248, mode = 1, s =   x, c = x
40ns :[DRV] a =122, b =   5, mode = 0, s =   x, c = x
50ns :[DRV] a =244, b = 236, mode = 0, s =   x, c = x
60ns :[DRV] a =236, b =  88, mode = 1, s =   x, c = x
70ns :[DRV] a = 68, b = 245, mode = 0, s =   x, c = x
80ns :[DRV] a =184, b =  53, mode = 1, s =   x, c = x
90ns :[DRV] a =211, b =  61, mode = 1, s =   x, c = x
100ns :[DRV] a =211, b =  88, mode = 1, s =   x, c = x
110ns :[DRV] a =130, b = 247, mode = 0, s =   x, c = x
```

3 gen.run() 종료

```
2480ns :[DRV] a =192, b = 182, mode = 1, s =   x, c = x
2490ns :[DRV] a = 87, b =  92, mode = 0, s =   x, c = x
2500ns :[DRV] a =141, b = 219, mode = 0, s =   x, c = x
2510ns :[DRV] a =128, b =  56, mode = 1, s =   x, c = x
2520ns :[DRV] a =  0, b = 118, mode = 1, s =   x, c = x
2530ns :[DRV] a =148, b =  14, mode = 1, s =   x, c = x
2540ns :[DRV] a =111, b =  37, mode = 0, s =   x, c = x
2550ns :[DRV] a = 75, b =  67, mode = 0, s =   x, c = x
exit
INFO: [Common 17-206] Exiting xsim at Thu Nov  5 08:57:33 2026...
~/.../lib/sim file$
```

forever 안 get() 대기